

A Aula 07 fala sobre expressões, atribuição e estruturas de controle. Ela mostra como as linguagens de programação decidem a ordem de avaliação das expressões, fazem conversões de tipos e controlam o fluxo de execução. O objetivo principal é entender porque um código pode produzir um resultado diferente daquilo que parece, evitando problemas de precedência, conversões implícitas, efeitos colaterais e estruturas de controle.

Inicialmente, a aula mostra as expressões aritméticas, explicando os conceitos de precedência e associatividade. A precedência indica qual operador será executado primeiro. A associatividade indica a ordem de execução quando operadores têm a mesma prioridade. Por exemplo, em $3 + 4 * 5$, a multiplicação ocorre antes da soma, resultando em 23. A ordem de avaliação dos operandos também importa, especialmente quando funções têm efeitos colaterais. Por isso, usar parênteses é recomendado quando houver dúvida.

Em seguida, a aula fala sobre efeitos colaterais e sobrecarga de operadores. Um efeito colateral acontece quando uma função, além de devolver um valor, altera alguma variável ou estado externo. Efeitos colaterais podem fazer o resultado depender da ordem de avaliação dos operandos. A sobrecarga permite que o mesmo operador tenha significados diferentes conforme os tipos envolvidos. O operador `+`, por exemplo, pode fazer soma numérica ou concatenar strings. Algumas linguagens também permitem que o programador defina comportamentos próprios para operadores.

Outro assunto importante são as conversões de tipos. A aula explica a diferença entre alargamento, quando um valor é convertido para um tipo que pode representar mais valores, e estreitamento, quando o oposto acontece. Algumas linguagens fazem essas conversões automaticamente; outras pedem que o programador faça explicitamente. Em Java, por exemplo, `double` pode ser convertido explicitamente para `int`, enquanto conversões de tipos menores para maiores podem acontecer automaticamente. O problema das conversões implícitas é que conversões implícitas podem esconder erros e gerar resultados inesperados.

JavaScript e PHP recebem atenção especial por usarem coerção implícita. Em JavaScript, `"5" - 2` dá 3, enquanto `"5" + 2` dá `"52"`, porque o operador pode provocar conversões diferentes conforme o contexto. A comparação `==` também pode converter os valores antes de compará-los, enquanto `===` exige que tipo e valor sejam iguais. Por isso, a aula mostra `===` e `!==` como alternativas para evitar problemas de coerção.

A aula também explica expressões relacionais e booleanas. As linguagens tratam valores booleanos de formas diferentes. Em C, por exemplo, zero significa falso e valores diferentes de zero significam verdadeiro. Assim, a expressão `3 > 2 > 1` dá um resultado que o programador pode não esperar, porque as comparações acontecem uma após a outra. Em Python, por outro lado, as comparações podem ser encadeadas, então `3 > 2 > 1` verifica ambas as relações. Java, C#, Go e Rust são mais rígidos e exigem valores booleanos nas condições.

Um conceito fundamental é a avaliação em curto-circuito. Em expressões com operadores como `&&` e `||`, nem todas as partes precisam ser avaliadas quando o resultado já pode ser decidido. A avaliação em curto-circuito ajuda a ganhar eficiência e a evitar erros, como acessar um objeto nulo. A aula também mostra recursos do JavaScript como `?.` e `??`, explicando que `||` e `??` não têm exatamente o mesmo comportamento.

Depois, o conteúdo passa para as sentenças de atribuição. Algumas linguagens permitem usar uma atribuição como expressão, como em C, C++ e JavaScript. Isso permite construções como `if (x = y)`, que compila mesmo quando o programador quis fazer uma comparação. Python tem recursos diferentes, como atribuição múltipla e o operador morsa `:=`. Go e Rust usam regras mais restritivas para evitar essa ambiguidade.

Também há uma armadilha importante envolvendo atribuição e conversão de tipos. A expressão `double taxa = acertos / total * 100` pode gerar `0.0` quando `acertos` e `total` são inteiros, porque a divisão inteira acontece antes que o resultado seja convertido para `double`. Por isso, é preciso garantir que a divisão seja feita com um tipo real.

Na segunda metade da aula, o foco muda para as estruturas de controle. As estruturas básicas são sequência, seleção e iteração. A sequência executa comandos em ordem. A seleção permite escolher entre caminhos diferentes, normalmente por `if` e `else`. A iteração permite repetir comandos enquanto uma condição for verdadeira. A aula destaca que essas estruturas são suficientes para representar qualquer algoritmo estruturado, enquanto estruturas como `for` e `switch` tornam o código mais fácil de escrever e ler.

Quando usamos `if` e `else`, surge o problema do `else` pendente. Em linguagens como C, Java, C++ e C#, o `else` sempre pertence ao `if` mais próximo que ainda não tem outro `else`, mesmo que a indentação pareça diferente. Por isso, usar chaves é essencial para deixar claro a que bloco cada condição pertence.

A aula também compara várias formas de seleção múltipla, como `switch` e `match`. Em C, se você esquecer o `break`, a execução pode continuar pelos próximos casos, algo chamado de `fall-through`. Java moderno tem uma versão de `switch` que funciona como expressão e impede esse problema. Rust usa `match`, que obriga a tratar todos os casos possíveis. Go e Python também têm suas próprias maneiras de fazer seleção múltipla.

Nos laços de repetição, a aula compara `for`, `while` e `do-while`. O `for` de C permite controlar inicialização, condição e incremento de forma explícita, enquanto o `for` de Python percorre diretamente um iterável. A aula também apresenta `break` e outras maneiras de interromper os laços. Um cuidado importante é evitar loops infinitos, certificando que as variáveis na condição de parada mudem de fato. O `do-while` difere do `while` porque executa o corpo pelo menos uma vez antes de checar a condição.

Por fim, a aula mostra goto e os comandos protegidos de Dijkstra. O goto faz um salto incondicional e costuma ser comentado por causa dos problemas de legibilidade e manutenção que gera. O exemplo do goto fail da Apple mostra como um erro simples no fluxo de execução pode causar problemas sérios. Os comandos protegidos de Dijkstra têm uma abordagem distinta: várias condições são avaliadas e, se mais de uma for verdadeira, uma alternativa pode ser escolhida de forma não determinística.

Em resumo, a ideia principal da aula é saber exatamente como um programa vai rodar. Para isso, precisamos entender a precedência dos operadores, a ordem de avaliação, os efeitos colaterais, a sobrecarga, as conversões de tipos, as regras booleanas, o curto-circuito, a atribuição e as estruturas de seleção e repetição. Os exercícios finais colocam esses conceitos em prática, mostrando erros reais como for...in no JavaScript, switch sem break, if com ;, sombreamento de variáveis e divisão inteira.